KAWASTOCK

Cahier des charges fonctionnel et technique

Application web open source de gestion de stock, d’achats et de ventes pour quincailleries et magasins de pièces détachées en Afrique francophone.

Backend	Django + Django REST Framework
Frontend	Vue 3 + Vite + Pinia + Vue Router
Base de données MVP	PostgreSQL (SQLite possible pour démarrer localement)
Orientation produit	MVP connecté, puis évolution vers un mode hors ligne synchronisé
Licence envisagée	AGPL-3.0, à valider avant publication
Version du document	1.0 — 24 octobre 2026

Document de référence : périmètre MVP, architecture, structure des applications, modèles de données, API, interfaces Vue, sécurité, stratégie de tests, déploiement, roadmap hors ligne et modèle économique.

Objectif : construire d’abord une solution utile et fiable, puis valider le marché avant d’investir dans les fonctions avancées.

Sommaire

- 1. Résumé exécutif et vision du produit
- 2. Contexte, problèmes et objectifs
- 3. Utilisateurs, rôles et permissions
- 4. Périmètre fonctionnel du MVP
- 5. Règles métier et exigences de gestion
- 6. Architecture technique
- 7. Structure des dépôts et applications
- 8. Modèles de données détaillés
- 9. API REST et contrats d’interface
- 10. Frontend Vue 3 : pages et composants
- 11. Sécurité, confidentialité et audit
- 12. Exigences non fonctionnelles
- 13. Tests et critères d’acceptation
- 14. Installation, déploiement et coûts de départ
- 15. Roadmap de développement du MVP
- 16. Suite hors ligne après le MVP
- 17. Stratégie de synchronisation et conflits
- 18. Modèle économique V1 / V2
- 19. Licence open source et gouvernance
- 20. Risques, décisions et critères de lancement
- Annexe A. Arborescence proposée
- Annexe B. Exemple de parcours de vente

Le document distingue explicitement le MVP connecté de la future version hors ligne. L’architecture initiale doit préparer cette évolution sans tenter de construire la synchronisation complète dès la première version.

1. Résumé exécutif et vision du produit

KawaStock est une application web de gestion commerciale destinée en priorité aux quincailleries et aux magasins de pièces détachées. Elle permet de centraliser les produits, les achats, les ventes, les mouvements de stock et les rapports, avec une interface française simple et adaptée aux petits commerces.

Le projet suit une stratégie open source : une édition Community gratuite, installable par le commerçant ou un prestataire, et une offre Pro payante qui vend principalement l'hébergement géré, les sauvegardes, l'assistance, la synchronisation et des fonctions avancées.

Principes directeurs

- Simplicité : les opérations fréquentes doivent être accessibles en peu d'étapes.
- Fiabilité : aucune vente confirmée ne doit entraîner un stock incohérent.
- Traçabilité : les corrections de stock et actions sensibles doivent être explicables.
- Adaptation locale : devise XOF/FCFA configurable, unités variées, interface française et faible consommation de données.
- Sécurité multi-commerce : un compte ne doit accéder qu'aux commerces auxquels il appartient.
- Évolutivité : concevoir le MVP pour permettre une future synchronisation hors ligne, sans en faire une promesse de la V1.

Indicateurs de réussite

Indicateur	Cible initiale à vérifier
Utilisabilité	Un vendeur réalise une vente simple après une courte prise en main.
Fiabilité stock	Aucune vente confirmée ne laisse le stock incohérent dans les tests.
Adoption pilote	3 à 5 commerces testent le MVP avec des données réelles consenties.
Usage	Les utilisateurs reviennent plusieurs jours par semaine.
Validation commerciale	Au moins un commerçant accepte de payer pour un service Pro ou une prestation.

2. Contexte, problèmes et objectifs

Problèmes ciblés

- Stock suivi dans des cahiers ou fichiers différents, difficiles à maintenir à jour.
- Recherche lente des références, variantes, marques ou codes-barres.
- Ruptures et surstocks faute de seuils d'alerte et de visibilité sur les ventes.
- Difficulté à distinguer chiffre d'affaires, coût des marchandises vendues et marge estimée.
- Écarts de stock non documentés et absence de journal d'actions.
- Connexion Internet parfois instable, à traiter dans une phase ultérieure hors ligne.

Objectifs du MVP

- Créer un catalogue structuré de produits, catégories, unités et références.
- Enregistrer les ventes avec calcul côté serveur et mise à jour transactionnelle du stock.
- Enregistrer les achats et réceptions.
- Afficher les produits en stock bas et les mouvements.
- Fournir des rapports de ventes et des exports CSV.
- Gérer plusieurs utilisateurs et plusieurs commerces avec séparation stricte des données.

Hors périmètre MVP

- Comptabilité légale, déclarations fiscales et facturation réglementaire certifiée.
- Intégration directe à Wave, banques ou terminaux de paiement.
- Synchronisation multi-appareils hors ligne et résolution automatique de conflits.
- Application native Android/iOS, IA, marketplace fournisseur et gestion avancée des livraisons.

- Gestion complète des réparations, garanties et compatibilités automobiles complexes.

Le MVP est une application web connectée. Si Internet est indisponible, les opérations serveur ne seront pas disponibles. L'interface doit afficher clairement l'état de connexion et éviter de faire croire qu'une vente a été enregistrée si le serveur ne l'a pas confirmée.

3. Utilisateurs, rôles et permissions

Rôle	Droits principaux	Restrictions
Propriétaire / Admin du commerce	Paramètres, utilisateurs, produits, achats, ventes, rapports, corrections et exports.	Uniquement son ou ses commerces autorisés.
Gestionnaire de stock	Produits, réceptions, mouvements, inventaires et fournisseurs.	Pas de gestion des abonnements ni des paramètres sensibles.
Vendeur	Recherche produit, création de vente, consultation de ses ventes selon politique définie.	Pas de modification libre des prix d'achat ni d'ajustement de stock.
Comptable / Lecture seule (option MVP)	Consultation des ventes et rapports autorisés.	Pas d'écriture ni de correction.
Administrateur plateforme (futur SaaS)	Support technique et gestion des comptes de service.	Accès aux données clients limité, tracé et soumis à une politique stricte.

Les rôles doivent être définis au niveau du commerce via ShopMembership, pas uniquement avec le superuser Django. Toutes les requêtes doivent filtrer par commerce autorisé.

4. Périmètre fonctionnel du MVP

4.1 Authentification et commerce

- Connexion et déconnexion ; renouvellement de session selon la stratégie retenue.
- Création du commerce et profil de base.
- Invitation ou création d'utilisateurs par le propriétaire.
- Rôle par utilisateur et par commerce.
- Devise par défaut XOF, fuseau horaire configurable et paramètres régionaux.
- Journal des actions sensibles.

4.2 Catalogue produits

- Créer, modifier, consulter, rechercher, filtrer et désactiver un produit.
- Champs : référence SKU, nom, description courte, marque, catégorie, code-barres facultatif, unité, prix d'achat, prix de vente, stock disponible, seuil minimum et statut actif.
- Empêcher les doublons de SKU au sein d'un même commerce.
- Prendre en charge pièce, boîte, paquet, mètre, rouleau, litre, kilogramme et unité personnalisée.
- Conserver un prix historique sur les lignes de vente et d'achat.

4.3 Stock et inventaire

- Entrées de stock via réception d'achat ou ajustement autorisé.
- Sorties via vente, retour fournisseur, perte ou correction documentée.
- Mouvements avec quantité signée ou type explicite, motif, utilisateur, date et référence source.
- Alerte lorsque stock disponible est inférieur ou égal au seuil minimum.
- Inventaire physique et ajustement avec justification.
- MVP : un seul emplacement logique par commerce ; multi-entrepôts en V2.

4.4 Achats et fournisseurs

- Créer et modifier un fournisseur avec nom, téléphone, adresse et notes.
- Créer une commande ou un achat en brouillon.
- Réceptionner des lignes d'achat et augmenter le stock uniquement lors de la réception validée.

- Conserver le coût unitaire et les quantités réellement reçues.
- Annulation et correction traçables ; pas de suppression silencieuse d’une réception validée.

4.5 Ventes

- Rechercher et ajouter des produits au panier.
- Valider quantité, prix autorisé, remise éventuelle et disponibilité.
- Calculer les totaux sur le serveur ; les montants envoyés par le client ne sont pas considérés comme fiables.
- Créer la vente et décrémenter le stock dans une transaction atomique.
- Enregistrer les lignes, prix appliqués, quantités, vendeur, date et statut.
- Gérer l’annulation par opération inverse et traçable, selon permissions.

4.6 Tableau de bord et rapports

- Ventes du jour et d’une période donnée.
- Nombre de ventes, montant brut, remises et montant net.
- Produits sous le seuil, produits les plus vendus et mouvements récents.
- Valeur du stock estimée à partir du coût retenu.
- Marge estimée uniquement si le coût des produits est correctement renseigné.
- Export CSV avec filtres de date et contrôle d’accès.

4.7 Historique et audit

- Journaliser les ajustements, annulations, changements de prix et changements de permissions.
- Conserver les références aux ventes et achats qui ont généré des mouvements.
- Ne pas exposer les secrets, jetons d’authentification ou données sensibles dans les logs.
- Définir une politique de conservation et d’export des journaux.

5. Règles métier et exigences de gestion

ID	Règle
BR-01	Chaque donnée métier appartient à un commerce. Les listes, détails, recherches et exports sont filtrés par autorisation.
BR-02	Le SKU est unique au sein d’un commerce ; les codes-barres peuvent être absents, mais doivent être validés contre les doublons selon la politique choisie.
BR-03	Les prix sont stockés en DecimalField, jamais en float. La devise et la règle d’arrondi sont explicites.
BR-04	Le stock ne peut pas devenir négatif, sauf décision métier future explicite. La validation doit être atomique.
BR-05	Une vente confirmée, ses lignes et ses mouvements de stock sont créés dans une transaction atomique.
BR-06	Les requêtes concurrentes sur le même produit doivent verrouiller ou mettre à jour la quantité de façon sûre.
BR-07	Les lignes de vente gardent le prix réellement appliqué et les informations de produit nécessaires à l’historique.
BR-08	Une opération validée n’est pas effacée silencieusement ; une annulation produit des mouvements inverses traçables.
BR-09	Le stock courant doit pouvoir être rapproché de l’historique des mouvements ; prévoir une commande de contrôle d’intégrité.
BR-10	Les calculs de chiffre d’affaires et de marge ont une définition documentée et testée.

Décision recommandée : utiliser StockMovement comme journal des changements et stock_quantity comme solde matérialisé pour la performance. Le solde doit être mis à jour dans la même transaction que le mouvement et contrôlé par des tests de cohérence.

6. Architecture technique

6.1 Architecture MVP

- Vue 3 + Vite : application monopage responsive.
- Django REST Framework : API JSON versionnée sous /api/v1/.
- PostgreSQL en environnement de test et production ; SQLite possible pour une première exploration locale.
- Authentification : privilégier des cookies de session sécurisés si le déploiement permet une même origine ; sinon JWT avec stockage et renouvellement conçus prudemment.
- CORS strictement limité aux origines autorisées.
- OpenAPI généré avec drf-spectacular pour documenter les contrats d'API.
- Tâches asynchrones, notifications et files de messages non indispensables au MVP.

6.2 Principes de séparation

- Les ViewSets et serializers gèrent la couche HTTP ; la logique de vente et de stock est isolée dans des services métier.
- Les modèles contiennent les contraintes et méthodes simples, mais ne doivent pas porter seuls toute la logique de transaction.
- Les permissions sont testées côté backend ; masquer un bouton Vue n'est pas une protection.
- Les filtres, la pagination et la recherche sont exécutés côté serveur pour éviter de télécharger tout le catalogue.
- La future synchronisation hors ligne utilisera une couche de commandes et des identifiants stables, décrits dans les sections 16 et 17.

6.3 Architecture logique

Couche	Responsabilité
UI Vue	Affichage, formulaires, validations ergonomiques, état local et retours utilisateur.
Services frontend	Appels API, gestion d'erreurs, conversion de dates et contrats de données.
API DRF	Authentification, permissions, sérialisation, pagination et validation d'entrée.
Services métier Django	Validation métier, transaction de vente, réception d'achat, ajustement et annulation.
Modèles / PostgreSQL	Contraintes, relations, index, verrouillage et stockage durable.
Audit / observabilité	Journal d'événements, logs techniques non sensibles et suivi d'erreurs.

7. Structure des dépôts et applications

Pour une seule personne, un monorepo simplifie la documentation et les versions. Le frontend et le backend restent des sous-projets séparés.

Applications Django recommandées

App	Responsabilité	Modèles principaux
config	Paramètres Django, URLs, ASGI/WSGI, configurations d'environnements.	Aucun modèle métier.
accounts	Profil utilisateur et authentification complémentaire.	User (custom recommandé dès le début), Profile si nécessaire.
shops	Commerces, adhésions et rôles.	Shop, ShopMembership.
catalog	Catalogue, catégories, marques et unités.	Category, Brand (facultatif), Unit (ou unité textuelle au MVP), Product.
inventory	Mouvements, inventaire, ajustements et alertes.	StockMovement, Stocktake, StocktakeLine (inventaire avancé éventuellement après MVP).
suppliers	Fournisseurs et contacts.	Supplier.
purchases	Achats, lignes et réceptions.	Purchase, PurchaseItem, GoodsReceipt (réception partielle si besoin).
sales	Ventes, lignes, annulations et paiements enregistrés manuellement.	Sale, SaleItem, SalePayment (facultatif au MVP).
reports	Agrégations et exports.	Pas de modèle au départ ; vues de rapport.
audit	Événements d'audit et historique de sécurité métier.	AuditEvent.
subscriptions (V2)	Plans, abonnements, limites et état de service.	Plan, Subscription, Entitlement.
sync (phase offline)	Appareils, opérations synchronisées et conflits.	Device, SyncOperation, SyncConflict.

Ne crée pas toutes les apps futures au premier jour. Pour le MVP, commence par config, accounts, shops, catalog, inventory, suppliers, purchases, sales et reports. Ajoute audit si le journal métier ne peut pas être correctement intégré à inventory/sales ; subscriptions et sync viendront plus tard.

8. Modèles de données détaillés

8.1 User

Champ	Type / contrainte	Description
id	UUID ou BigAutoField	Identifiant interne.
username / email	Unique selon politique choisie	Connexion et contact.
first_name, last_name	CharField facultatif	Nom d'affichage.
is_active	Boolean	Compte actif.
date_joined	DateTime	Date de création.

Recommandation : définir AUTH_USER_MODEL avant la première migration, même si le modèle personnalisé ne contient au début que peu de champs. Utiliser UUID pour les entités qui devront être créées hors ligne est utile, mais pas obligatoire pour tous les modèles du MVP.

8.2 Shop

Champ	Type / contrainte	Description
id	UUID recommandé	Identifiant du commerce.
name	CharField(180)	Nom affiché.
slug	SlugField	Identifiant lisible facultatif.
currency	CharField(3), défaut XOF	Devise de référence.
timezone	CharField, défaut Africa/Abidjan	Fuseau horaire.
is_active	Boolean	État du commerce.
created_at / updated_at	DateTime	Suivi temporel.

8.3 ShopMembership

Champ	Type / contrainte	Description
shop	FK Shop	Commerce concerné.
user	FK User	Utilisateur membre.
role	Choice	owner, manager, stock_manager, seller, viewer.
is_active	Boolean	Accès actif ou suspendu.
created_at	DateTime	Date d’adhésion.
Contrainte	Unique(shop, user)	Une adhésion active logique par utilisateur et commerce.

8.4 Category et Product

Modèle / champ	Type / règle	Description
Category.shop	FK Shop	Propriétaire de la catégorie.
Category.name	CharField(120)	Nom de catégorie.
Category.parent	FK nullable (optionnel)	Sous-catégorie future.
Product.shop	FK Shop	Commerce propriétaire.
Product.sku	CharField(80)	Référence interne, unique par commerce.
Product.barcode	CharField(100), blank	Code-barres facultatif.
Product.name	CharField(200)	Nom commercial.
Product.brand	CharField ou FK facultative	Marque ou fabricant.
Product.unit	CharField ou FK Unit	pièce, mètre, boîte, etc.
Product.cost_price	Decimal(12,2)	Coût d’achat courant de référence.
Product.selling_price	Decimal(12,2)	Prix de vente par défaut.
Product.stock_quantity	Decimal(12,3)	Solde courant matérialisé.
Product.min_stock	Decimal(12,3)	Seuil d’alerte.
Product.is_active	Boolean	Désactivation sans effacer l’historique.
Product.created_at / updated_at	DateTime	Audit temporel de base.

Pour les pièces détachées automobiles, une future extension pourra gérer références fabricant, équivalences, modèles compatibles et numéros OEM. Ne transforme pas cette extension en dépendance du MVP.

8.5 StockMovement

Champ	Type / règle	Description
id	UUID recommandé	Identifiant stable, utile pour le mode offline.
shop	FK Shop	Commerce concerné.
product	FK Product	Produit affecté.
movement_type	Choice	purchase_receipt, sale, sale_return, supplier_return, adjustment, loss, transfer.
quantity_delta	Decimal(12,3)	Variation signée du stock.
quantity_before / after	Decimal(12,3), facultatif	Soldes de contrôle et audit.
source_type / source_id	Référence métier	Vente, réception ou ajustement source.
reason	Text facultatif	Motif de l'opération.
created_by	FK User nullable	Auteur.
created_at	DateTime	Horodatage serveur.

Éviter les références polymorphes trop libres si elles rendent l'intégrité difficile à garantir. Au besoin, garder des FK explicites ou un identifiant de source accompagné d'un type validé dans le service métier.

8.6 Supplier

Champ	Type / règle	Description
shop	FK Shop	Commerce propriétaire.
name	CharField(180)	Nom du fournisseur.
phone	CharField(40), blank	Téléphone.
email	EmailField, blank	Email.
address	TextField, blank	Adresse.
notes	TextField, blank	Notes internes.
is_active	Boolean	Fournisseur actif.

8.7 Purchase et PurchaseItem

Modèle / champ	Type / règle	Description
Purchase.shop	FK Shop	Commerce.
Purchase.supplier	FK Supplier	Fournisseur.
Purchase.status	Choice	draft, ordered, partially_received, received, cancelled.
Purchase.reference	CharField	Référence interne unique par commerce.
Purchase.total_amount	Decimal(14,2)	Total calculé côté serveur.
PurchaseItem.purchase	FK Purchase	Entête d'achat.
PurchaseItem.product	FK Product	Produit acheté.
PurchaseItem.quantity_ordered	Decimal(12,3)	Quantité commandée.
PurchaseItem.quantity_received	Decimal(12,3)	Quantité reçue.
PurchaseItem.unit_cost	Decimal(12,2)	Coût unitaire historique.

Pour simplifier le MVP, une commande peut être réceptionnée en une seule opération. Les réceptions partielles et le modèle GoodsReceipt deviennent nécessaires si les commerçants en ont réellement besoin.

8.8 Sale et SaleItem

Modèle / champ	Type / règle	Description
Sale.shop	FK Shop	Commerce.
Sale.sold_by	FK User	Vendeur.
Sale.number	Référence unique par commerce	Numéro lisible de vente.
Sale.status	Choice	draft, completed, cancelled, refunded (politique à définir).
Sale.subtotal	Decimal(14,2)	Sous-total calculé.
Sale.discount_amount	Decimal(14,2)	Remise validée.
Sale.total_amount	Decimal(14,2)	Total net calculé.
Sale.created_at	DateTime	Date de création.
SaleItem.sale	FK Sale	Vente parente.
SaleItem.product	FK Product	Produit vendu.
SaleItem.product_name_snapshot	CharField	Nom au moment de la vente, recommandé.
SaleItem.sku_snapshot	CharField	Référence au moment de la vente.
SaleItem.quantity	Decimal(12,3)	Quantité vendue.
SaleItem.unit_price	Decimal(12,2)	Prix appliqué.
SaleItem.unit_cost_snapshot	Decimal(12,2)	Coût retenu pour calcul de marge.
SaleItem.line_total	Decimal(14,2)	Total calculé.

Une stratégie de coût doit être choisie et documentée : coût courant, coût moyen pondéré ou autre méthode. Le MVP peut utiliser un coût de référence simple pour une marge indicative, mais ne doit pas prétendre produire une valorisation comptable certifiée.

8.9 AuditEvent

Champ	Type / règle	Description
shop	FK Shop nullable	Commerce concerné si applicable.
actor	FK User nullable	Utilisateur à l'origine.
action	CharField / Choice	Ex. product.updated, sale.cancelled, stock.adjusted.
entity_type / entity_id	Référence	Entité affectée.
metadata	JSONField limité	Champs non sensibles nécessaires à l'audit.
created_at	DateTime	Horodatage serveur.

Ne pas enregistrer de mot de passe, jeton, numéro de carte ou données personnelles inutiles dans metadata.

8.10 Relations et contraintes

- Shop 1—N Product, Category, Supplier, Sale, Purchase et StockMovement.
- User N—N Shop via ShopMembership, avec un rôle par commerce.
- Sale 1—N SaleItem ; Purchase 1—N PurchaseItem.
- Product 1—N SaleItem, PurchaseItem et StockMovement.
- Contraintes uniques composites pour SKU et références de vente/achat au niveau du commerce.
- Index sur shop, date, SKU, barcode, product et status pour les requêtes fréquentes.
- Utiliser PROTECT ou désactivation logique pour les produits référencés par des opérations validées.

9. API REST et contrats d'interface

9.1 Convention générale

- Préfixe versionné : /api/v1/.
- JSON en entrée et sortie ; erreurs structurées et compréhensibles.
- Pagination serveur pour les listes.
- Filtres explicites : search, category, is_active, low_stock, date_from, date_to.
- Codes HTTP standards : 200, 201, 204, 400, 401, 403, 404, 409 et 422 si convention documentée.
- La documentation OpenAPI doit être générée et mise à jour avec l'API.

Méthode / route	Usage
POST /auth/token/ ou /auth/login/	Authentification selon le mécanisme retenu.
GET, POST /shops/	Lister ou créer les commerces autorisés selon le parcours de compte.
GET, PATCH /shops/{id}/	Lire et modifier les paramètres autorisés.
GET, POST /products/	Lister et créer les produits.
GET, PATCH /products/{id}/	Consulter ou modifier un produit.
GET, POST /categories/	Lister et créer les catégories.
GET, POST /suppliers/	Lister et créer les fournisseurs.
GET, POST /purchases/	Lister et créer les achats.
POST /purchases/{id}/receive/	Valider une réception et créer les mouvements.
GET, POST /sales/	Lister les ventes et créer une vente.
POST /sales/{id}/cancel/	Annuler une vente avec mouvement inverse autorisé.
GET /stock/movements/	Consulter les mouvements filtrés.
POST /stock/adjustments/	Ajustement documenté du stock.

Méthode / route	Usage
GET /dashboard/summary/	Indicateurs synthétiques.
GET /reports/sales/	Rapport par période.
GET /reports/low-stock/	Liste des produits sous le seuil.
GET /reports/export/?format=csv	Export autorisé et filtré.

9.2 Exemple de requête de vente

```
POST /api/v1/sales/ { "items": [ {"product_id": "UUID-PRODUIT-1", "quantity": "2"}, {"product_id": "UUID-PRODUIT-2", "quantity": "1.5"} ], "discount_amount": "0.00", "idempotency_key": "UUID-UNIQUE-OPERATION" }
```

Le serveur détermine le commerce à partir de l'adhésion active de l'utilisateur, charge les produits de ce commerce, vérifie les quantités et permissions, calcule les prix, verrouille ou met à jour les lignes concernées, crée la vente et les mouvements puis renvoie le résultat.

9.3 Idempotence

Prévoir un identifiant d'opération unique pour les créations de vente. Si un client réessaie une requête après une coupure réseau, le serveur doit éviter de créer deux ventes identiques. Au MVP connecté, cette protection est utile ; elle devient essentielle en phase hors ligne.

9.4 Erreurs API

- Retourner un message par champ pour les erreurs de validation.
- Ne jamais exposer de stack trace en production.
- Utiliser 409 Conflict pour les conflits de stock ou de version lorsque pertinent.
- Le frontend doit distinguer une vente refusée d'une vente dont le résultat est inconnu à cause d'une coupure.

10. Frontend Vue 3 : pages et composants

Page	Composants / besoins	Priorité
LoginView	Formulaire, erreurs, état de chargement.	MVP
DashboardView	KPI, graphiques simples, alertes et dernières ventes.	MVP
ProductListView	Table responsive, recherche, filtres, pagination.	MVP
ProductFormView	Validation, unité, catégorie, prix et seuil.	MVP
SaleCreateView	Recherche rapide, panier, quantités, total et confirmation.	MVP
SalesHistoryView	Filtres de date, détails, statut et annulation autorisée.	MVP
PurchaseListView / Form	Fournisseurs, lignes et réception.	MVP
StockView	Mouvements, alertes, ajustements et motif obligatoire.	MVP
SupplierView	Liste, création, modification et désactivation.	MVP
ReportsView	Rapports filtrés et export CSV.	MVP
UsersView	Invitations et rôles du commerce.	MVP ou juste après MVP
SettingsView	Nom du commerce, devise, fuseau horaire.	MVP
SyncCenterView	État des opérations locales, conflits et dernière synchronisation.	Phase offline
DevicesView	Appareils autorisés et révocation.	Phase offline / Pro

Principes UX

- Interface mobile-first avec navigation claire et actions principales visibles.
- Recherche rapide par SKU, nom et code-barres.
- Confirmations explicites pour annulation, ajustement et désactivation.
- Indiquer les erreurs au bon endroit et préserver les données d'un formulaire en cas d'échec.
- Ne pas afficher un succès avant la confirmation serveur.
- Afficher date, heure et devise selon les paramètres du commerce.
- Accessibilité : contrastes lisibles, labels de formulaire et navigation clavier.

Stores Pinia

- authStore : utilisateur courant, état de connexion et commerce actif.
- shopStore : paramètres du commerce et rôle actif.
- productStore facultatif : cache temporaire des listes ; ne pas dupliquer toute la logique métier.
- Pour le mode offline, ajouter outboxStore et syncStore dans une phase distincte.

11. Sécurité, confidentialité et audit

- Configurer HTTPS en production et les cookies Secure, HttpOnly et SameSite lorsque les sessions par cookie sont utilisées.
- Si JWT est retenu, définir une stratégie de stockage, de renouvellement, d'expiration et de révocation ; éviter de stocker des jetons durables dans localStorage sans analyse de risque.
- Protéger les routes DRF par authentification et permissions explicites.
- Tester systématiquement l'accès croisé : un utilisateur du commerce A ne doit jamais lire ou modifier les données du commerce B.
- Valider toutes les entrées côté serveur ; les validations Vue améliorent l'UX mais ne remplacent pas celles du backend.
- Utiliser des variables d'environnement pour les secrets ; ne jamais committer .env.
- Limiter la taille des fichiers si des imports sont ajoutés, et vérifier le contenu des fichiers.
- Mettre en place sauvegardes, tests de restauration et procédure de gestion d'incident.
- Limiter les données personnelles collectées et documenter leur durée de conservation.
- Envisager les obligations applicables en Côte d'Ivoire, notamment celles relatives aux données personnelles, avant le lancement commercial.

Pour les ventes concurrentes, utiliser transaction.atomic et une stratégie de verrouillage ou de mise à jour conditionnelle au niveau de la base. Un simple test Python « si stock suffisant » sans verrouillage peut permettre à deux ventes simultanées de consommer la même quantité.

12. Exigences non fonctionnelles

Domaine	Exigence
Performance	Listes paginées ; index adaptés ; pas de chargement complet du catalogue.
Fiabilité	Transactions atomiques pour vente, réception et annulation.
Maintenabilité	Apps Django cohérentes, services métier testables et conventions documentées.
Portabilité	Installation documentée sur environnement local Linux/Windows via instructions claires ou Docker facultatif.
Responsive	Utilisable sur smartphone, tablette et ordinateur.
Observabilité	Logs techniques non sensibles, erreurs suivies et health check simple.
Sauvegarde	Procédure de sauvegarde et restauration documentée avant le pilote.
Accessibilité	Formulaires labellisés, contrastes, navigation clavier.
Localisation	Français, XOF configurable et fuseau horaire du commerce.
Extensibilité	API versionnée, identifiants stables et séparation des services métier.

13. Tests et critères d’acceptation

Tests backend indispensables

- Un utilisateur non connecté ne peut pas accéder aux données privées.
- Un utilisateur ne peut pas lire, modifier, exporter ou supprimer les données d’un autre commerce.
- Le SKU dupliqué est refusé dans un commerce mais accepté dans un autre.
- Une vente avec stock insuffisant est refusée sans écriture partielle.
- Une vente valide crée la vente, les lignes et les mouvements en une transaction.
- Deux ventes simultanées ne peuvent pas faire passer le stock sous zéro.
- Une annulation produit des mouvements inverses cohérents et ne peut pas être appliquée deux fois.
- Une réception validée augmente le stock exactement une fois.
- Les totaux sont calculés par le serveur et les prix client non autorisés sont ignorés.
- Les filtres de rapports et les exports respectent les permissions et la période.

Tests frontend indispensables

- Formulaires avec champs obligatoires, erreurs API et état de chargement.
- Le panier recalcule correctement les quantités et affiche les erreurs serveur.
- Un échec réseau ne présente pas la vente comme réussie.
- La pagination, la recherche et les filtres restent utilisables sur mobile.
- Le routeur protège les pages privées sans remplacer les permissions backend.

Critères de sortie du MVP

- Les parcours produit → achat/réception → vente → rapport fonctionnent.
- Les tests critiques de permissions et de stock passent.
- Les migrations sont reproductibles sur une base vide.
- Un guide d’installation et un jeu de données de démonstration sont disponibles.
- Les sauvegardes et la restauration ont été testées.
- Au moins trois utilisateurs pilotes ont essayé le produit et les problèmes bloquants sont corrigés.

14. Installation, déploiement et coûts de départ

Développement local

- Installer Python, Node.js, Git et PostgreSQL ; utiliser des versions supportées et les consigner dans le README.
- Créer un environnement virtuel Python et installer les dépendances depuis requirements.txt ou pyproject.toml.
- Configurer les variables d'environnement dans un fichier local non versionné.
- Créer la base, lancer les migrations et créer un compte administrateur.
- Démarrer Django et Vite séparément en développement.
- Exécuter les tests backend et frontend avant chaque fusion de code.

Déploiement pilote

- Choisir un hébergeur gratuit disponible à la date de lancement, en vérifiant stockage, veille, quotas, sauvegardes et conditions.
- Ne pas dépendre d'un hébergement gratuit comme garantie de disponibilité commerciale.
- Séparer les environnements développement, test et production.
- Activer HTTPS, secrets d'environnement, journaux et sauvegardes.
- Préparer une exportation complète des données pour limiter le verrouillage fournisseur.

Le développement peut commencer sans dépense. Un service SaaS fiable engendre toutefois des coûts récurrents à mesure que les utilisateurs, les données et les exigences de disponibilité augmentent. Les prix et limites d'hébergement doivent être vérifiés au moment du déploiement.

15. Roadmap de développement du MVP

Étape	Travail	Livrable
S1 — Fondations	Dépôt, environnements, custom User, Shop, membership, auth et permissions.	Connexion et isolation des commerces.
S2 — Catalogue	Catégories, produits, recherche, unités, validation et pages Vue.	CRUD produits fonctionnel.
S3 — Stock	StockMovement, ajustements, seuils, fournisseurs et réception.	Stock traçable.
S4 — Ventes	Panier, Sale/SaleItem, transaction, annulation et idempotence.	Parcours de vente fiable.
S5 — Rapports	Dashboard, filtres, CSV, tests et documentation API.	MVP complet sur environnement de test.
S6 — Pilote	Sécurité, sauvegarde, déploiement, formation de 3–5 commerces.	Retour terrain et liste de priorités.

Le calendrier est indicatif. Il faut prioriser les parcours critiques et éviter de commencer l'abonnement, les paiements ou l'offline tant que le MVP n'a pas été testé.

16. Suite hors ligne après le MVP

Objectif de la phase 2 : permettre à un magasin de continuer certaines opérations lorsque la connexion Internet est coupée, puis de synchroniser les changements lorsque le réseau revient. Cette fonction doit être ajoutée après validation du MVP connecté.

16.1 Approche recommandée

Transformer progressivement le frontend Vue en Progressive Web App (PWA), avec un service worker pour les ressources de l'application et IndexedDB pour les données locales. Le backend Django/PostgreSQL reste la source centrale pour les données synchronisées. Le mode hors ligne doit être explicite et limité aux opérations conçues pour fonctionner localement.

- PWA installable : manifest, icônes, cache des fichiers statiques et page de reprise hors connexion.

- IndexedDB via une bibliothèque maintenue, par exemple Dexie, pour le cache du catalogue et les opérations en attente.
- Outbox locale : chaque action hors ligne est enregistrée comme une opération unique, avec UUID, date locale, appareil et charge utile minimale.
- Centre de synchronisation : en attente, synchronisé, erreur ou conflit.
- Synchronisation en arrière-plan si le navigateur le supporte, et bouton de synchronisation manuelle dans tous les cas.
- Le serveur valide les opérations et retourne un résultat durable ; le client ne doit pas considérer une opération comme synchronisée avant cet accusé de réception.

16.2 Fonctionnalités hors ligne à livrer progressivement

Lot	Fonctions offline	Difficulté
O1 — Lecture	Ouvrir l'app, consulter un catalogue récemment synchronisé et voir la date de mise à jour.	Faible à moyenne
O2 — Brouillons	Préparer un panier ou une commande en brouillon sans valider la vente.	Moyenne
O3 — Ventes offline	Enregistrer localement une vente avec identifiant unique et la mettre en attente de synchronisation.	Élevée
O4 — Multi-appareils	Synchroniser plusieurs appareils et résoudre les conflits de stock.	Très élevée
O5 — Gestion avancée	Transferts, inventaires, import/export offline et règles multi-sites.	Très élevée

La recommandation est de livrer O1 puis O2, tester la fiabilité et seulement ensuite autoriser les ventes hors ligne. Un panier hors ligne ne doit pas être confondu avec une vente confirmée.

16.3 Modèles et champs à prévoir pour la synchronisation

Entité	Champs additionnels ou modèle futur	Utilité
Entités synchronisables	UUID stable, updated_at, version/revision si nécessaire.	Identifier sans collision et détecter des changements.
Device	id UUID, shop, user, device_name, last_seen_at, revoked_at.	Identifier et révoquer un appareil.
SyncOperation	operation_id UUID, device_id, shop_id, operation_type, payload, local_created_at, server_received_at, status.	Dédupliquer, auditer et traiter les commandes.
SyncConflict	operation, entity_type, entity_id, conflict_type, resolution_status, resolution_data.	Rendre les conflits visibles et décidables.
Outbox locale	operation_id, operation_type, payload, created_at, retry_count, state.	Garder les opérations en attente dans IndexedDB.

16.4 Décisions d'architecture à préparer dès le MVP

- Utiliser des UUID pour les entités qui pourraient être créées localement hors ligne.
- Ne pas dépendre de séquences entières attribuées uniquement par le serveur comme identifiants d'entités.
- Centraliser les règles de vente dans des services métier réutilisables.
- Prévoir une clé d'idempotence pour les créations de vente.
- Ne pas considérer l'heure du téléphone comme une source fiable pour l'ordre des événements.
- Conserver les mouvements de stock comme journal explicatif.
- Documenter la différence entre opération créée localement, acceptée par le serveur et définitivement synchronisée.

16.5 Limites et risques

- IndexedDB et le cache navigateur peuvent être effacés ; l'utilisateur doit être averti que les opérations non synchronisées ne sont pas encore sauvegardées sur le serveur.

- Le navigateur peut suspendre les tâches en arrière-plan : la synchronisation manuelle reste nécessaire.
- Un service worker ne rend pas automatiquement toute l'application offline.
- Les données locales doivent être protégées sur les appareils partagés et l'accès doit être révocable.
- Ne pas stocker plus de données personnelles que nécessaire sur les appareils.
- La synchronisation doit survivre aux retries, redémarrages et coupures en cours d'envoi.

17. Stratégie de synchronisation et conflits

17.1 Règle fondamentale

Le stock est une donnée partagée et mutable. Si deux appareils hors ligne vendent le dernier article, aucun algorithme de synchronisation ne peut garantir après coup que les deux ventes étaient réellement couvertes par le stock. Il faut définir une politique métier claire et l'expliquer au commerçant.

17.2 Stratégie recommandée

- Chaque opération offline possède un UUID unique et une clé d'idempotence.
- À la synchronisation, le serveur authentifie l'appareil, vérifie son accès au commerce et valide l'opération.
- Le serveur vérifie le stock actuel et la politique choisie avant d'accepter une vente.
- Les opérations sont traitées de façon idempotente : un même operation_id ne crée pas plusieurs ventes.
- Le serveur retourne accepted, rejected ou conflict avec une raison et un identifiant de ressource.
- Les opérations rejetées restent visibles pour une résolution manuelle ; elles ne sont pas supprimées silencieusement.
- Les modifications de catalogue peuvent souvent suivre une stratégie serveur-prioritaire ; les ventes et mouvements de stock nécessitent une logique métier dédiée.

17.3 Politiques de stock offline possibles

Politique	Avantages	Inconvénients
Réservation préalable	Réduit le risque de double vente ; stock offline plafonné par appareil.	Nécessite une connexion préalable et la gestion des réservations expirées.
Acceptation différée avec vérification	Simple à comprendre et à mettre en place.	Une vente peut être rejetée après coup si le stock est insuffisant.
Autoriser un stock négatif temporaire	Évite de bloquer le vendeur en zone sans réseau.	Risque opérationnel ; exige rapprochement et règles de correction explicites.

Pour une première version commerciale, je recommande la réservation préalable ou l'acceptation différée avec vérification, et non le stock négatif silencieux. La politique finale doit être choisie avec les commerçants pilotes.

17.4 Flux de synchronisation

- 1. L'appareil récupère un snapshot ou les changements depuis le serveur.
- 2. L'utilisateur effectue une opération locale autorisée.
- 3. L'application écrit l'opération dans l'outbox IndexedDB.
- 4. Quand le réseau revient, le client envoie les opérations non acquittées.
- 5. Le serveur déduplique, valide les permissions et applique chaque opération en transaction.
- 6. Le serveur renvoie le statut durable et les nouvelles versions.
- 7. Le client marque les opérations acquittées et récupère les changements serveur.
- 8. Les conflits sont affichés pour résolution ; aucune donnée en attente n'est effacée automatiquement.

17.5 Critères de sortie de la phase offline

- L'application s'ouvre sans réseau après une synchronisation préalable.
- Les opérations locales survivent à la fermeture et réouverture du navigateur.
- Une même opération envoyée plusieurs fois ne crée qu'un seul résultat serveur.
- Les conflits de stock sont détectés et affichés avec une action explicite.
- Un appareil révoqué ne peut plus synchroniser de nouvelles opérations.

- Les tests couvrent les coupures pendant envoi, les retries et les opérations concurrentes.
- La documentation explique les limites du mode hors ligne et le risque lié à un appareil perdu.

18. Modèle économique V1 / V2

V1 Community gratuite

- Code source sous licence open source choisie.
- Catalogue, achats, ventes, mouvements, alertes et rapports simples.
- Installation autonome documentée.
- Support communautaire et exemples de données.

V2 Pro hébergée

- Hébergement géré et mises à jour.
- Sauvegardes automatiques et restauration assistée.
- Gestion multi-utilisateurs et rapports avancés.
- Synchronisation multi-appareils et fonctions hors ligne une fois stabilisées.
- Support, formation, import de catalogues et accompagnement.

Offre hypothétique	Prix de départ à tester	Valeur vendue
Community	0 FCFA	Logiciel libre, installation autonome.
Pro	10 000–20 000 FCFA / mois	Hébergement, sauvegarde, support et gestion d'équipe.
Business	25 000–40 000 FCFA / mois	Plusieurs sites, rapports consolidés et support prioritaire.
Installation / import	10 000–25 000 FCFA ponctuels	Configuration, migration Excel et formation.

Ces prix sont des hypothèses à tester et ne sont pas des tarifs de marché vérifiés. Avant d'investir dans le cloud, interroger les clients sur leur volonté de payer, leurs contraintes de connexion et le coût d'une erreur de stock.

Stratégie de validation

- Mener 5 à 10 entretiens avant de développer des fonctions avancées.
- Lancer un pilote auprès de 3 à 5 commerces.
- Mesurer l'usage hebdomadaire, le temps gagné, les erreurs évitées et les fonctions demandées.
- Proposer un service d'installation ou de migration avant de bâtir un système de paiement intégré.
- Ne lancer les abonnements automatisés qu'après validation des besoins, du support et des coûts réels.

19. Licence open source et gouvernance

Licence envisagée : GNU AGPL-3.0. Elle est adaptée si l'objectif est de préserver la disponibilité du code source des versions modifiées mises à disposition sur un réseau, sous réserve de respecter les conditions de la licence.

- Lire la licence officielle et demander un avis juridique si le modèle devient commercial.
- Inclure le fichier LICENSE, les notices de copyright et les notices des dépendances.
- Ne pas supposer que la licence empêche un tiers de vendre un service basé sur le code.
- Documenter la procédure de contribution, les tests requis et les conventions de code.
- Protéger les secrets, données clients et fichiers de configuration ; ne jamais les publier dans Git.
- Si une double licence est envisagée, vérifier que tous les contributeurs ont accordé les droits nécessaires.

Référence officielle : https://www.gnu.org/licenses/agpl-3.0.html

Gouvernance Git

- main : version stable ; develop facultatif si le flux de travail le justifie.
- Branches courtes feature/* et bugfix/*.

- Pull requests même en solo pour relire les changements importants.
- Tests automatiques et formatage/linting avant fusion.
- CHANGELOG, versions taguées et notes de version.

20. Risques, décisions et critères de lancement

Risque	Réponse recommandée
Périmètre trop large	Limiter le MVP aux produits, achats, ventes, stock et rapports simples.
Mauvaise qualité des données	Importer avec prévisualisation, validation et sauvegarde avant toute migration.
Stock incorrect	Transactions atomiques, mouvements traçables, tests de concurrence et inventaires.
Fuite inter-commerces	Permissions DRF, filtrage serveur systématique et tests négatifs.
Hébergement gratuit instable	Distinguer démo et production ; prévoir sauvegarde/export et budget dès les clients payants.
Clients non prêts à payer	Entretiens, pilotes, service d'installation et mesure de la valeur avant le développement Pro.
Mode offline trop complexe	Le repousser après le MVP ; commencer par lecture offline et brouillons.
Conflits de stock offline	Politique explicite, idempotence, vérification serveur et résolution visible.

Décisions à prendre avant le code

- Un commerce correspond-il toujours à un seul magasin dans le MVP ? Recommandation : oui.
- Les ventes à crédit sont-elles dans le MVP ? Recommandation : enregistrer d'abord le mode de paiement simple, repousser le vrai suivi des créances si le délai est serré.
- La remise est-elle autorisée aux vendeurs ? Recommandation : limite paramétrable et permission dédiée.
- La réception d'achat est-elle partielle ? Recommandation : une réception simple d'abord.
- La marge est-elle indicative ou basée sur un coût moyen ? Recommandation : documenter la méthode et éviter les promesses comptables.
- Offline : le commerce préfère-t-il bloquer, réserver ou accepter avec risque de rejet ? Décider avec les utilisateurs pilotes.

Définition de terminé

Une fonctionnalité est terminée lorsque ses règles métier sont documentées, les permissions testées, les erreurs affichées correctement, les migrations vérifiées, les tests critiques passés et la documentation utilisateur mise à jour.

Annexe A. Arborescence proposée

```
kawastock/
■■■ README.md
■■■ LICENSE
■■■ .gitignore
■■■ docs/
■ ■■■ cahier-des-charges.md
■ ■■■ architecture.md
■ ■■■ api.md
■ ■■■ installation.md
■ ■■■ decisions/
■ ■ ■■■ 0001-stack.md
■ ■ ■■■ 0002-offline-later.md
■ ■■■ offline/
■ ■■■ sync-protocol.md
■ ■■■ conflict-policy.md
■■■ backend/
■ ■■■ manage.py
■ ■■■ pyproject.toml
■ ■■■ .env.example
■ ■■■ config/
■ ■ ■■■ settings/
■ ■ ■ ■■■ base.py
■ ■ ■ ■■■ local.py
■ ■ ■ ■■■ production.py
■ ■ ■■■ urls.py
■ ■ ■■■ asgi.py
■ ■ ■■■ wsgi.py
■ ■■■ accounts/
■ ■■■ shops/
■ ■■■ catalog/
■ ■■■ inventory/
■ ■■■ suppliers/
■ ■■■ purchases/
■ ■■■ sales/
■ ■■■ reports/
■ ■■■ audit/
■ ■■■ tests/
■■■ frontend/
■ ■■■ package.json
■ ■■■ vite.config.js
■ ■■■ .env.example
■ ■■■ src/
■ ■■■ assets/
■ ■■■ components/
■ ■■■ layouts/
■ ■■■ router/
■ ■■■ services/
■ ■■■ stores/
■ ■■■ views/
■ ■■■ main.js
■■■ deployment/
```

■■■ `docker-compose.yml (facultatif)`

■■■ `backend.Dockerfile (facultatif)`

■■■ `frontend.Dockerfile (facultatif)`

Exemples de modules par app

- sales/services.py : create_sale(), cancel_sale(), calculate_totals().
- inventory/services.py : receive_stock(), adjust_stock(), create_movement().
- shops/permissions.py : membership_required(), role_required().
- catalog/filters.py : ProductFilter avec recherche, catégorie et low_stock.
- reports/services.py : agrégations et calculs documentés.
- sync/ (future) : apply_operation(), idempotency.py, conflict_resolution.py.

Annexe B. Exemple de parcours de vente

- 1. Le vendeur se connecte et sélectionne le commerce autorisé.
- 2. Il recherche une référence SKU ou scanne un code-barres.
- 3. Il ajoute la quantité au panier ; le frontend affiche un total provisoire.
- 4. Il confirme la vente ; le frontend envoie les produits et quantités, plus une clé d’idempotence.
- 5. Le backend vérifie l’adhésion, les permissions, les produits, les prix et le stock.
- 6. Dans une transaction atomique, le backend crée la vente, les lignes et les mouvements, puis met à jour les soldes.
- 7. Le serveur répond avec le numéro de vente et le total calculé.
- 8. Le frontend affiche le succès seulement après la confirmation serveur.
- 9. Si le résultat réseau est inconnu, le client vérifie l’opération par sa clé d’idempotence avant de réessayer.
- 10. En phase offline, les étapes 4 à 8 seront remplacées par une opération locale en attente puis une confirmation de synchronisation distincte.

Conclusion

KawaStock doit d’abord prouver qu’il rend la gestion quotidienne d’une quincaillerie plus fiable. Le MVP connecté construit les fondations : données propres, permissions, mouvements de stock et ventes transactionnelles. La phase hors ligne doit ensuite s’appuyer sur des identifiants stables, une outbox locale, l’idempotence et une politique explicite de conflits.

La meilleure stratégie commerciale consiste à garder l’édition Community utile et ouverte, puis à vendre la fiabilité opérationnelle : hébergement, sauvegarde, migration, formation, support et synchronisation. Le produit doit être validé sur le terrain avant d’ajouter des fonctions coûteuses.